Dossier de Cadrage Fonctionnel & Architectural

Sujet : Passerelle de Notifications Omnicanale, Apatride (Stateless) et Résiliente avec Moteur de Routage Dynamique

Ce document sert de base de réflexion et d'alignement avec les exigences académiques en Génie Logiciel.

1. Alignement Académique (Méthode Houndji)

• **Le Problème Concret :** L'indisponibilité imprévisible des API de communication tierces (Twilio, Resend, Firebase) qui impacte la distribution des messages critiques (2FA, transactions) et la dispersion de la logique de routage dans le code des applications clientes.

• **La Solution / L'Apport :** Une infrastructure intermédiaire unifiée (DevTool) qui abstrait ces services, gère la tolérance aux pannes de manière invisible (Failover), et permet de modifier les stratégies de routage sans retoucher au code de production.

• **La Preuve attendue :** Scénarios d'évaluation par injection de fautes (Chaos Engineering) : coupure volontaire du fournisseur principal et mesure fine de la latence de bascule et du taux d'échec résiduel.

2. Spécifications du Moteur de Routage Statless

Pour garantir la conformité RGPD par le design (Privacy by Design) et maximiser le débit d'ingestion, le système n'enregistre aucune donnée sur les destinataires finaux. Toutes les variables d'évaluation doivent être fournies à la volée dans la charge utile (payload).

Exemple de Payload JSON pour l'API unifiée (POST /api/v1/send) :

```
{
  "channels": ["sms", "email"],
  "content": "Votre code temporaire est 849201",
  "targets": {
    "sms": "+33612345678",
    "email": "user@example.com"
  },
  "context": {
    "country": "FR",
    "priority": "high"
  }
}
```

3. Logique Spécifique des Règles et Conditions

Le système évalue les règles selon une hiérarchie stricte d'interception :

1. **Surcharges explicites (Overrides) :** Présentes directement dans le payload de la requête pour forcer un canal ou un identifiant de fournisseur.

2. **Règles conditionnelles dynamiques :** Évaluées par le Rule Engine en faisant correspondre l'objet `context` reçu avec les schémas JSON configurés sur le Dashboard utilisateur (Ex: filtrage géographique par pays ou par niveau de priorité).

3. **Configuration globale par défaut :** Fournisseurs de secours universels configurés par canal pour le compte client si aucune règle spécifique ne correspond.

4. Architecture Technique et Gestion de la Tolérance aux Pannes

L'architecture repose sur le découpage asynchrone pour ne pas bloquer l'application appelante :

• **API Gateway (Ingestion rapide) :** Reçoit la requête, valide la clé d'API, charge la configuration de routage, pousse la tâche dans Redis, et retourne un statut `202 Accepted` en moins de 10ms.

• **Message Queue (Redis + BullMQ/Go Channels) :** Tamponne les demandes pour absorber les pics de charge et éviter les dénis de service internes.

• **Workers & Pattern Adapter :** Processus d'arrière-plan qui instancient l'adaptateur requis (ex: TwilioAdapter, FCMAdapter) via une interface standardisée. En cas d'erreur HTTP ou de timeout du fournisseur tiers, le worker intercepte l'exception, incrémente les tentatives, et exécute automatiquement la boucle sur le fournisseur de secours (Chain of Responsibility).

5. Viabilité Économique (Modèle BYOP - Bring Your Own Provider)

• **Faibles coûts d'exploitation :** L'absence de stockage de données lourdes simplifie l'infrastructure. Un simple serveur VPS couplé à une base PostgreSQL/Redis managée (< 50$/mois) peut router des millions de messages.

• **Monétisation :** Facturation à la consommation du volume d'appels d'API (Modèle SaaS pur) et à l'accès aux fonctionnalités d'infrastructure critique (moteur de bascule automatique, historique étendu des logs d'erreurs).

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.